

CSED 312 Project 2 Design Report

김지현(20220302)

전현빈(20220259)

Analyze the Current Implementation

1. Process execution

Pintos가 실행된 후 user program이 어떻게 시작되는지 확인하기 위해, Pintos의 main entry point를 확인해보았다. 아래 코드에서 `#ifdef USERPROG` 부분을 보았을 때, `USERPROG` 매크로가 정의된 경우에만 user program과 관련된 초기화 코드가 컴파일된다는 것을 확인했다.

```
int
main (void)
{
    ...

    /* Segmentation. */
#ifdef USERPROG
    tss_init ();
    gdt_init ();
#endif

    /* Initialize interrupt handlers. */
    intr_init ();
    timer_init ();
    kbd_init ();
    input_init ();
#ifdef USERPROG
    exception_init ();
    syscall_init ();
#endif
    ...
}
```

user program 초기화 과정을 확인하기 위해 먼저 `tss_init()` 함수와 `gdt_init()` 함수에 대해 알아보았다. TSS구조체와 이를 초기화하는 `tss_init()`은 `~src/userprog/tss.c`에 구현되어 있다. 먼저, TSS의 구조체를 설명하자면, TSS는 "Task State Segment"를 정의하는 구조체로, kernel mode와 user mode 사이에서 context switch가 일어날 때 필요하다. 각 context 상태인 레지스터 값, 스택 포인터 등을 저장하고 복원하는 데 사용된다. 현재 사용되는 x86 OS에서는 이러한 TSS 기능이 거의 사용되지 않지만, user mode에서 interrupt가 발생하여 kernel mode로 전환될 때는 TSS가 사용된다.

```
struct tss
{
    uint16_t back_link, :16;
    void *esp0;                /**< Ring 0 stack virtual address. */
    uint16_t ss0, :16;         /**< Ring 0 stack segment selector. */
```

```

    void *esp1;
    uint16_t ss1, :16;
    void *esp2;
    uint16_t ss2, :16;
    uint32_t cr3;
    void (*eip) (void);
    uint32_t eflags;
    uint32_t eax, ecx, edx, ebx;
    uint32_t esp, ebp, esi, edi;
    uint16_t es, :16;
    uint16_t cs, :16;
    uint16_t ss, :16;
    uint16_t ds, :16;
    uint16_t fs, :16;
    uint16_t gs, :16;
    uint16_t ldt, :16;
    uint16_t trace, bitmap;
};

```

위는 tss structure의 구현이다. tss는 각 task의 status와 관련된 정보를 저장하고 관리한다. `back_link`는 이전 task의 tss segment selector로서 task간의 linked list data structure에서 previous task로의 pointer이다. `esp0`는 kernel mode에서 사용하는 stack pointer로서 kernel mode로 전환될 때 사용하는 stack의 주소를 저장한다. `ss0`는 stack segment selector로서 kernel mode에서 사용되는 stack segment를 정의한다. `cr3`는 PageDirectoryBaseRegister의 주소로 페이지 디렉토리의 physical adress를 저장하여 memory management와 adress 변환에 사용된다. `eip`는 next instruction의 주소이며 `eax`, `ecx`, 등은 범용 레지스터이다. 이같은 tss 구조체는 하나의 task의 상태 정보를 저장하여 새로운 태스크의 상태를 restore하거나 current task의 정보를 저장한다. 이는 kernel이 task switching을 하는 과정에서 register, segment 정보를 손실하지 않도록 돕는다.

main 함수에서 사용된 `tss_init` 함수는 위에서 소개한 TSS를 초기화하는 함수였다. 먼저 `palloc`을 사용해 TSS를 위한 메모리를 할당했다. 그리고 kernel mode에서 사용하는 stack segment selector인 `ss0`을 `SEL_KDSEG`로 초기화해주었다. `SEL_KDSEG`는 kernel data segment를 가리킨다고 한다. 그 다음, I/O 접근을 허용하거나 제한할 때 사용하는 bitmask인 `bitmap`을 `0xdfff`로 초기화해주었다. 이후 아래에서 설명한 `tss_update` 함수를 통해 현재 TSS의 `esp0` 값을 업데이트해주었다. `esp0`은 kernel mode stack pointer로, user mode에서 kernel mode로 전환될 때 사용할 stack 위치를 설정해준다. kernel mode에서는 stack이 위에서 아래로 쌓이기 때문에 `esp0`에는 현재 thread 주소에 `PGSIZE`를 더해 최상단 주소를 저장해주었다.

```

void
tss_init (void)
{
    /* Our TSS is never used in a call gate or task gate, so only a
       few fields of it are ever referenced, and those are the only
       ones we initialize. */
    tss = palloc_get_page (PAL_ASSERT | PAL_ZERO);
    tss->ss0 = SEL_KDSEG;
    tss->bitmap = 0xdfff;
    tss_update ();
}

void
tss_update (void)

```

```
{
    ASSERT (tss != NULL);
    tss->esp0 = (uint8_t *) thread_current () + PGSIZE;
}
```

추가적으로, tss_get 함수는 현재 TSS의 포인터를 반환하는 역할을 하는 함수이다.

```
struct tss *
tss_get (void)
{
    ASSERT (tss != NULL);
    return tss;
}
```

TSS를 초기화해준 다음 gdt를 초기화해주기 때문에, 이번에는 gdt에 대해서 알아보았다. GDT (Global Descriptor Table)는 code, data, 그리고 TSS로 구성된 메모리 세그먼트를 저장하는 테이블이다.

~src/userprog/gdt.c에 구현된 gdt_init 함수를 보면, 이후에 설명할 함수들을 이용해 kernel의 code, data 등을 테이블에 추가하는 것을 볼 수 있었다. 먼저 null을 넣는데, 이는 CPU가 잘못된 세그먼트를 참조할 때를 대비해 null을 가장 먼저 추가한 것이다. 그 다음으로, kernel code segment, kernel data segment, user code segment, user data segment, 마지막으로 TSS를 추가해주었다. 이후 해당 GDT의 크기와 주소를 저장하는 GDTR (GDT register) 값을 저장한 다음, 어셈블리 명령어를 통해 CPU가 GDT와 TSS를 알 수 있도록 했다. 이 함수는 아래에서 다시 설명할 것이다.

```
static uint64_t gdt[SEL_CNT];

/** Sets up a proper GDT. The bootstrap loader's GDT didn't
    include user-mode selectors or a TSS, but we need both now. */
void
gdt_init (void)
{
    uint64_t gdtr_operand;

    /* Initialize GDT. */
    gdt[SEL_NULL / sizeof *gdt] = 0;
    gdt[SEL_KCSEG / sizeof *gdt] = make_code_desc (0);
    gdt[SEL_KDSEG / sizeof *gdt] = make_data_desc (0);
    gdt[SEL_UCSEG / sizeof *gdt] = make_code_desc (3);
    gdt[SEL_UDSEG / sizeof *gdt] = make_data_desc (3);
    gdt[SEL_TSS / sizeof *gdt] = make_tss_desc (tss_get ());

    /* Load GDTR, TR. See [IA32-v3a] 2.4.1 "Global Descriptor
        Table Register (GDTR)", 2.4.4 "Task Register (TR)", and
        6.2.4 "Task Register". */
    gdtr_operand = make_gdtr_operand (sizeof gdt - 1, gdt);
    asm volatile ("lgdt %0" : : "m" (gdtr_operand));
    asm volatile ("ltr %w0" : : "q" (SEL_TSS));
}
```

위의 설명에서 kernel/user code segment와 data segment를 정의하는 함수들로 make_seg_desc 함수를 사용해 GDT에 추가될 entry를 생성했다. 여기서 함수 인자로 dpl(descriptor privilege level)을 받아오는데, 이는 각각 kernel(0)과 user(3) mode를 확인하기 위한 것이다.

```
static uint64_t
make_code_desc (int dpl)
{
    return make_seg_desc (0, 0xffffffff, CLS_CODE_DATA, 10, dpl, GRAN_PAGE);
}

static uint64_t
make_data_desc (int dpl)
{
    return make_seg_desc (0, 0xffffffff, CLS_CODE_DATA, 2, dpl, GRAN_PAGE);
}
```

make_seg_desc 함수를 보면, 입력으로 받아오는 base, limit, class type, dpl, granularity를 GDT에 들어갈 64비트로 저장해 반환하는 역할을 했다. 32비트인 e0과 e1에 나눠서 저장한 다음, 둘을 합쳐 64비트로 반환했다. e0에는 limit의 하위 16비트와 base의 하위 16비트를 저장했고, e1에는 base의 중간 8비트, type, class, dpl, limit의 상위 4비트, granularity, 그리고 base의 상위 8비트를 저장했다. 여기서 말하는 type 은 segment 의 유형을 나타내는 변수로 read/write 이 가능한지 등을 알려준다. 그리고 class 는 segment 의 클래스로 code/data 인지 system 인지를 확인할 때 필요하다.

```
enum seg_class
{
    CLS_SYSTEM = 0,          /**< System segment. */
    CLS_CODE_DATA = 1       /**< Code or data segment. */
};

enum seg_granularity
{
    GRAN_BYTE = 0,          /**< Limit has 1-byte granularity. */
    GRAN_PAGE = 1          /**< Limit has 4 kB granularity. */
};

static uint64_t
make_seg_desc (uint32_t base,
               uint32_t limit,
               enum seg_class class,
               int type,
               int dpl,
               enum seg_granularity granularity)
{
    uint32_t e0, e1;

    ASSERT (limit <= 0xffffffff);
    ASSERT (class == CLS_SYSTEM || class == CLS_CODE_DATA);
    ASSERT (type >= 0 && type <= 15);
    ASSERT (dpl >= 0 && dpl <= 3);
```

```

    ASSERT (granularity == GRAN_BYTE || granularity == GRAN_PAGE);

    e0 = ((limit & 0xffff)           /**< Limit 15:0. */
        | (base << 16));           /**< Base 15:0. */

    e1 = (((base >> 16) & 0xff)      /**< Base 23:16. */
        | (type << 8)               /**< Segment type. */
        | (class << 12)             /**< 0=system, 1=code/data. */
        | (dpl << 13)               /**< Descriptor privilege. */
        | (1 << 15)                 /**< Present. */
        | (limit & 0xf0000)          /**< Limit 16:19. */
        | (1 << 22)                 /**< 32-bit segment. */
        | (granularity << 23)        /**< Byte/page granularity. */
        | (base & 0xff000000));      /**< Base 31:24. */

    return e0 | ((uint64_t) e1 << 32);
}

```

그리고 TSS를 GDT에 넣을 때, 아래의 make_tss_desc 함수를 통해 추가했다. 이 함수에서도 위에서 설명한 make_seg_desc 함수를 통해 entry를 생성했다.

```

static uint64_t
make_tss_desc (void *laddr)
{
    return make_seg_desc ((uint32_t) laddr, 0x67, CLS_SYSTEM, 9, 0, GRAN_BYTE);
}

```

make_gdtr_operand 함수는 GDT register 를 가져오는 함수로 GDT 의 크기인 limit 과 시작 주소인 base 를 반환 해준다.

```

static uint64_t
make_gdtr_operand (uint16_t limit, void *base)
{
    return limit | ((uint64_t) (uint32_t) base << 16);
}

```

/src/userprg/process.c 파일에는 사용자 프로그램 실행을 위한 프로세스 생성, 시작, 종료 등의 함수들이 정의되어 있다. 먼저, process_execute 함수는 인자로 전달받은 file_name을 이용해 새로운 스레드를 생성하고, 이후 start_process 함수가 실행되도록 한다.

```

tid_t
process_execute (const char *file_name)
{
    char *fn_copy;
    tid_t tid;

    /* Make a copy of FILE_NAME.

```

```

    Otherwise there's a race between the caller and load(). */
    fn_copy = palloc_get_page (0);
    if (fn_copy == NULL)
        return TID_ERROR;
    strncpy (fn_copy, file_name, PGSIZE);

    /* Create a new thread to execute FILE_NAME. */
    tid = thread_create (file_name, PRI_DEFAULT, start_process, fn_copy);
    if (tid == TID_ERROR)
        palloc_free_page (fn_copy);
    return tid;
}

```

위에서 `thread_create`를 통해 생성된 스레드는 `start_process` 함수를 실행하게 되며, 이 함수는 사용자 프로그램을 로드하고 실행하는 역할을 한다. 이후 설명할 `load` 함수를 통해 실행 파일이 메모리에 로드된 후, 사용자 모드로 전환되어 프로그램이 진행된다.

```

static void
start_process (void *file_name_)
{
    char *file_name = file_name_;
    struct intr_frame if_;
    bool success;

    /* Initialize interrupt frame and load executable. */
    memset (&if_, 0, sizeof if_);
    if_.gs = if_.fs = if_.es = if_.ds = if_.ss = SEL_UDSEG;
    if_.cs = SEL_UCSEG;
    if_.eflags = FLAG_IF | FLAG_MBS;
    success = load (file_name, &if_.eip, &if_.esp);

    /* If load failed, quit. */
    palloc_free_page (file_name);
    if (!success)
        thread_exit ();

    /* Start the user process by simulating a return from an
       interrupt, implemented by intr_exit (in
       threads/intr-stubs.S). Because intr_exit takes all of its
       arguments on the stack in the form of a `struct intr_frame',
       we just point the stack pointer (%esp) to our stack frame
       and jump to it. */
    asm volatile ("movl %0, %%esp; jmp intr_exit" : : "g" (&if_) : "memory");
    NOT_REACHED ();
}

```

`process_wait` 함수는 특정 스레드의 종료를 기다린 후 해당 스레드의 종료 상태를 반환하는 함수지만, 현재는 -1을 반환하도록 구현되어 있다. 아래의 design proposal 에서 이 함수의 수정 사항을 설명할 것이다.

```
int
process_wait (tid_t child_tid UNUSED)
{
    return -1;
}
```

process_exit 함수는 스레드가 종료될 때 호출되어, 현재 프로세스의 자원을 메모리에서 해제하고 커널 페이지 디렉터리로 전환하는 역할을 한다. 먼저, 현재 스레드의 페이지 디렉터를 null로 설정하여 인터럽트가 발생하더라도 해당 페이지 디렉터를 사용할 수 없도록 하고, 이후 커널 모드로 올바르게 전환되도록 페이지 디렉터를 활성화한다. 마지막으로 null로 설정된 페이지 디렉터를 메모리에서 해제한다. 이 순서로 진행해야 인터럽트 발생 시 문제를 방지하고, 메모리를 올바르게 해제할 수 있다.

```
void
process_exit (void)
{
    struct thread *cur = thread_current ();
    uint32_t *pd;

    /* Destroy the current process's page directory and switch back
       to the kernel-only page directory. */
    pd = cur->pagedir;
    if (pd != NULL)
    {
        /* Correct ordering here is crucial. We must set
           cur->pagedir to NULL before switching page directories,
           so that a timer interrupt can't switch back to the
           process page directory. We must activate the base page
           directory before destroying the process's page
           directory, or our active page directory will be one
           that's been freed (and cleared). */
        cur->pagedir = NULL;
        pagedir_activate (NULL);
        pagedir_destroy (pd);
    }
}
```

다음으로, process_activate 함수는 컨텍스트 스위치가 발생할 때 현재 스레드의 페이지 디렉터를 활성화하고 TSS를 업데이트해 준다.

```
void
process_activate (void)
{
    struct thread *t = thread_current ();

    /* Activate thread's page tables. */
    pagedir_activate (t->pagedir);

    /* Set thread's kernel stack for use in processing
```

```

    interrupts. */
    tss_update ();
}

```

load 함수는 앞서 설명한 start_process에서 호출되어 ELF 실행 파일을 로드하는 역할을 한다. 페이지 디렉터리를 생성한 후, process_activate 함수를 통해 해당 프로세스의 페이지 디렉터리를 활성화한다. 이어서, 인자로 전달된 파일 이름에 해당하는 파일을 열고 ELF 헤더를 읽어온다. 이후 if 문을 통해 ELF 포맷에 맞는지 확인하고, 문제가 없다면 for 루프를 통해 프로그램 헤더를 읽고 각 세그먼트를 로드한다. 그런 다음, 아래에서 설명할 setup_stack 함수를 호출해 사용자 모드에서 실행될 스택을 초기화하고 스택 포인터를 설정한다. 마지막으로 프로그램의 시작 주소를 eip 포인터가 가리키도록 설정한 후 파일을 닫고 성공 여부를 반환한다.

```

bool
load (const char *file_name, void (**eip) (void), void **esp)
{
    struct thread *t = thread_current ();
    struct Elf32_Ehdr ehdr;
    struct file *file = NULL;
    off_t file_ofs;
    bool success = false;
    int i;

    /* Allocate and activate page directory. */
    t->pagedir = pagedir_create ();
    if (t->pagedir == NULL)
        goto done;
    process_activate ();

    /* Open executable file. */
    file = filesys_open (file_name);
    if (file == NULL)
    {
        printf ("load: %s: open failed\n", file_name);
        goto done;
    }

    /* Read and verify executable header. */
    if (file_read (file, &ehdr, sizeof ehdr) != sizeof ehdr
        || memcmp (ehdr.e_ident, "\177ELF\1\1\1", 7)
        || ehdr.e_type != 2
        || ehdr.e_machine != 3
        || ehdr.e_version != 1
        || ehdr.e_phentsize != sizeof (struct Elf32_Phdr)
        || ehdr.e_phnum > 1024)
    {
        printf ("load: %s: error loading executable\n", file_name);
        goto done;
    }

    /* Read program headers. */
    file_ofs = ehdr.e_phoff;
    for (i = 0; i < ehdr.e_phnum; i++)

```



```

{
    struct Elf32_Phdr phdr;

    if (file_ofs < 0 || file_ofs > file_length (file))
        goto done;
    file_seek (file, file_ofs);

    if (file_read (file, &phdr, sizeof phdr) != sizeof phdr)
        goto done;
    file_ofs += sizeof phdr;
    switch (phdr.p_type)
    {
        case PT_NULL:
        case PT_NOTE:
        case PT_PHDR:
        case PT_STACK:
        default:
            /* Ignore this segment. */
            break;
        case PT_DYNAMIC:
        case PT_INTERP:
        case PT_SHLIB:
            goto done;
        case PT_LOAD:
            if (validate_segment (&phdr, file))
            {
                bool writable = (phdr.p_flags & PF_W) != 0;
                uint32_t file_page = phdr.p_offset & ~PGMASK;
                uint32_t mem_page = phdr.p_vaddr & ~PGMASK;
                uint32_t page_offset = phdr.p_vaddr & PGMASK;
                uint32_t read_bytes, zero_bytes;
                if (phdr.p_filesz > 0)
                {
                    /* Normal segment.
                       Read initial part from disk and zero the rest. */
                    read_bytes = page_offset + phdr.p_filesz;
                    zero_bytes = (ROUND_UP (page_offset + phdr.p_memsz, PGSIZE)
                                - read_bytes);
                }
                else
                {
                    /* Entirely zero.
                       Don't read anything from disk. */
                    read_bytes = 0;
                    zero_bytes = ROUND_UP (page_offset + phdr.p_memsz, PGSIZE);
                }
                if (!load_segment (file, file_page, (void *) mem_page,
                                read_bytes, zero_bytes, writable))
                    goto done;
            }
            else
                goto done;
            break;
    }
}

```

```

    }

    /* Set up stack. */
    if (!setup_stack (esp))
        goto done;

    /* Start address. */
    *eip = (void (*) (void)) ehdr.e_entry;

    success = true;

done:
    /* We arrive here whether the load is successful or not. */
    file_close (file);
    return success;
}

```

위에서 설명한 바와 같이, `setup_stack` 함수는 사용자 모드에서 사용할 스택을 `palloc`을 통해 메모리에 할당한 후, 스택 포인터(`esp`)가 `PHYS_BASE`를 가리키도록 설정한다. `PHYS_BASE`는 메모리의 가장 상단 주소로, 스택은 상단에서 아래로 확장되며 사용된다. 이후, `install_page` 함수의 성공 여부를 반환한다.

```

static bool
setup_stack (void **esp)
{
    uint8_t *kpage;
    bool success = false;

    kpage = palloc_get_page (PAL_USER | PAL_ZERO);
    if (kpage != NULL)
    {
        success = install_page (((uint8_t *) PHYS_BASE) - PGSIZE, kpage, true);
        if (success)
            *esp = PHYS_BASE;
        else
            palloc_free_page (kpage);
    }
    return success;
}

```

지금부터는 앞서 설명한 process 관련 함수들이 실제 유저프로그램의 실행과정에서 어떻게 사용되는지에 대해 알아보겠다. user program의 entry point는 `~src/lib/user/entry.c`에 구현된 `_start`이다.

```

// entry.c
...

void
_start (int argc, char *argv[])
{
    exit (main (argc, argv));
}

```

```
}
...

```

`_start`는 `argc`, `argv`를 입력받아 `main`에 이를 전달하고 `main`이 `return`하면 `exit`을 호출하여 종료한다. 이때 `argv`는 `main`함수가 전달받은 인자 각각이며 `argc`는 이러한 `argv`의 수이다. `main`함수는 이처럼 `argv`, `argc`를 전달받고 적절한 초기화 과정을 거친 후 `run_action(argv)`를 통해 `argv`를 실행하고 종료한다.

```
// init.c
// int main(void)
...

printf ("Boot complete.\n");

/* Run actions specified on kernel command line. */
run_actions (argv);

/* Finish up. */
shutdown ();
thread_exit ();

...

```

동일한 `init.c`에 구현된 `run_actions`에서는 전달받은 `argv`에 관한 정보를 아래와 같이 `action`이라는 구조체로 관리한다.

```
// init.c
// static void run_actions(char **argv)
...

struct action
{
    char *name;                /* Action name. */
    int argc;                  /* # of args, including action name. */
    void (*function) (char **argv); /* Function to execute action. */
};

static const struct action actions[] =
{
    {"run", 2, run_task},
#ifdef FILESYS
    /* 이 부분은 FILESYS에 대한 부분이므로 생략한다. */
#endif
    {NULL, 0, NULL},
};

...

```

입력된 명령이 run인 경우라면, action구조체는 `run_task`에 `argv`를 전달하여 실행한다. `run_task`또한 동일한 `init.c`에 구현되어있다.

```
/* Runs the task specified in ARGV[1]. */
static void
run_task (char **argv)
{
    const char *task = argv[1];

    printf ("Executing '%s':\n", task);
#ifdef USERPROG
    process_wait (process_execute (task));
#else
    run_test (task);
#endif
    printf ("Execution of '%s' complete.\n", task);
}
```

pintos 주석에 쓰여진 바와 같이, run task는 `argv[argc]`에 저장된 명령중 `argv[1]`을 시행한다. 이때 `argv[0]`은 run 이기 때문이다.

System call

system call은 user program과 kernel간의 상호작용이자 user program의 의도적인 kernel의 동작 수행의 요청으로 해석할 수 있다. 따라서 system call은 memory에서 user virtual memory와 kernel virtual memory간의 상호작용을 필수적으로 수반한다. user virtual memory adress의 범위는 0부터 `PHYS_BASE`까지로 주어지며 이러한 virtual adress는 process 단위로 주어지고 user program의 `PHYS_BASE` 너머의 kernel virtual memory adress의 access가 발생할경우 page fault가 발생한다. 각 process 마다의 고유한 pointer가 있어 process와 그것의 page가 mapping되고 각 page와 실제 physical memory는 page table의 정의에 따라 mapping된다. user program이 사용하는 user virtual memory는 user stack, heap, data, text로 구성된다. `PHYS_BASE`부터는 kernel virtual memory가 고유한 1GB의 크기를 차지한다. pintos guide에 따르면 user virtual memory와는 달리 kernel virtual memory의 경우 모든 process가 하나의 단일한 physical memory와 direct mapping되어 있으며 이때 kernel virtual memory adress의 `PHYS_BASE`는 0x0지점으로 mapping된다. 예를들어, `src/tests/userprog`의 user program에서 `syscall`이 요청되면 `src/lib/user/syscall.c`의 `syscall`함수(e.g, `halt`)가 호출된다. 이는 argument의 수에 따라 `syscall0`, `syscall1`, ..., `syscall13`중 하나로 arguments를 전달하고 호출하며 이는 어셈블리어를 통해 system call number를 stack에 push하여 kernel로 하여금 system call number를 처리할 수 있도록 저장하고 `int $0x30`을 이용해 pintos system call interrupt를 처리하도록 한다. 요컨대, `syscallN`은 N개의 arguments와 system call number를 전달받아 이를 스택에 저장하고 kernel에 전달함으로써 해당 system call을 호출하고 처리된 후 `%eax`에 반환된 결과를 `retval`에 저장하여 반환하도록 한다. 예시로서 아래에 `syscall0`의 code implement를 보였다.

```
/* Invokes syscall NUMBER, passing no arguments, and returns the
   return value as an `int'. */
#define syscall0(NUMBER) \
    ({ \
        \
```

```

        int retval;                                \
        asm volatile                                \
        ("pushl %[number]; int $0x30; addl $4, %%esp" \
         : "=a" (retval)                            \
         : [number] "i" (NUMBER)                    \
         : "memory");                               \
        retval;                                     \
    })

```

int는 pintos에서 지정된 system call을 invoke하는 instruction으로서 system call number와 arguments들이 user stack에 push 되고 `int $0x30`이 invoke되면 interrupt가 발생한다. 이는 src/threads/intr-stubs.S에 구현된바와 같이 src/threads/interrupt.c의 `intr_handler`를 호출시키며 `intr_handler`는 이어서 특정 interrupt를 handle 할 수 있는 함수를 call한다. 즉 설명하고 있는 system call의 interrupt의 경우는 `syscall_handler`을 호출한다.

앞서 exception_init()과 syscall_init() 함수에 대해 알아보았다. 먼저, 아래의 syscall_init 함수를 보면 intr_register_init 함수를 호출해 syscall interrupt를 등록했다. 그리고 intr_register_int 함수를 호출했다.

```

void
syscall_init (void)
{
    intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");
}

```

intr_register_init 함수는 vector number, dpl, level, handler 포인터, 그리고 이름을 argument로 받아온 다음, 유효한 벡터 값인지 확인한 후 다시 register_handler 함수로 변수들을 넘겨주었다. 이 intr_register_init 함수는 아래에서 설명할 exception_init에서도 사용된다.

register_handler에서는 interrupt 혹은 exception이 발생했을 때, 각 vector number에 맞는 interrupt/exception handler를 IDT (Interrupt Descriptor Table)에 등록하는 함수였다. 만약 interrupt가 활성화된 상태에서 처리해야 한다면 make_trap_gate 함수를 통해 IDT 엔트리를 설정하고, 그렇지 않다면 make_intr_gate 함수를 통해 CPU가 interrupt를 비활성화한 상태에서 handler를 실행한다.

```

void
intr_register_int (uint8_t vec_no, int dpl, enum intr_level level,
                  intr_handler_func *handler, const char *name)
{
    ASSERT (vec_no < 0x20 || vec_no > 0x2f);
    register_handler (vec_no, dpl, level, handler, name);
}

static void
register_handler (uint8_t vec_no, int dpl, enum intr_level level,
                 intr_handler_func *handler, const char *name)
{
    ASSERT (intr_handlers[vec_no] == NULL);
    if (level == INTR_ON)
        idt[vec_no] = make_trap_gate (intr_stubs[vec_no], dpl);
}

```

```

else
    idt[vec_no] = make_intr_gate (intr_stubs[vec_no], dpl);
    intr_handlers[vec_no] = handler;
    intr_names[vec_no] = name;
}

```

위에서 설명했듯이, `make_trap_gate`와 `make_intr_gate`는 interrupt 활성화/비활성화의 차이를 가진다. 하지만 내부 로직을 보면, 두 함수 모두 `make_gate` 함수를 호출한다. `make_gate` 함수는 위에서 넘겨준 인자들로 64비트인 gate descriptor를 생성해 반환해준다. 이 descriptor는 IDT에 추가될 것이다.

```

static uint64_t
make_trap_gate (void (*function) (void), int dpl)
{
    return make_gate (function, dpl, 15);
}

static uint64_t
make_intr_gate (void (*function) (void), int dpl)
{
    return make_gate (function, dpl, 14);
}

static uint64_t
make_gate (void (*function) (void), int dpl, int type)
{
    uint32_t e0, e1;

    ASSERT (function != NULL);
    ASSERT (dpl >= 0 && dpl <= 3);
    ASSERT (type >= 0 && type <= 15);

    e0 = (((uint32_t) function & 0xffff)      /**< Offset 15:0. */
          | (SEL_KCSEG << 16));              /**< Target code segment. */

    e1 = (((uint32_t) function & 0xffff0000) /**< Offset 31:16. */
          | (1 << 15)                        /**< Present. */
          | ((uint32_t) dpl << 13)           /**< Descriptor privilege level. */
          | (0 << 12)                        /**< System. */
          | ((uint32_t) type << 8));         /**< Gate type. */

    return e0 | ((uint64_t) e1 << 32);
}

```

Pintos에서 system call이 발생하면, 아래의 `syscall_handler` 함수가 실행된다. 현재 함수를 보면, 단순히 system call이 발생했다는 것을 알리기 위해 `printf`를 사용해 메시지를 출력하고, `thread_exit` 함수를 이용해 해당 thread를 종료해주었다.

```

static void
syscall_handler (struct intr_frame *f UNUSED)

```

```
{
    printf ("system call!\n");
    thread_exit ();
}
```

아래의 exception_init 함수를 보면, 위에서 설명한 intr_register_init 함수를 이용해 각 exception을 처리하는 handler들을 IDT에 추가하는 역할을 한다.

```
void
exception_init (void)
{
    /* These exceptions can be raised explicitly by a user program,
       e.g. via the INT, INT3, INTO, and BOUND instructions.  Thus,
       we set DPL==3, meaning that user programs are allowed to
       invoke them via these instructions. */
    intr_register_int (3, 3, INTR_ON, kill, "#BP Breakpoint Exception");
    intr_register_int (4, 3, INTR_ON, kill, "#OF Overflow Exception");
    intr_register_int (5, 3, INTR_ON, kill,
                      "#BR BOUND Range Exceeded Exception");

    /* These exceptions have DPL==0, preventing user processes from
       invoking them via the INT instruction.  They can still be
       caused indirectly, e.g. #DE can be caused by dividing by
       0. */
    intr_register_int (0, 0, INTR_ON, kill, "#DE Divide Error");
    intr_register_int (1, 0, INTR_ON, kill, "#DB Debug Exception");
    intr_register_int (6, 0, INTR_ON, kill, "#UD Invalid Opcode Exception");
    intr_register_int (7, 0, INTR_ON, kill,
                      "#NM Device Not Available Exception");
    intr_register_int (11, 0, INTR_ON, kill, "#NP Segment Not Present");
    intr_register_int (12, 0, INTR_ON, kill, "#SS Stack Fault Exception");
    intr_register_int (13, 0, INTR_ON, kill, "#GP General Protection Exception");
    intr_register_int (16, 0, INTR_ON, kill, "#MF x87 FPU Floating-Point Error");
    intr_register_int (19, 0, INTR_ON, kill,
                      "#XF SIMD Floating-Point Exception");

    /* Most exceptions can be handled with interrupts turned on.
       We need to disable interrupts for page faults because the
       fault address is stored in CR2 and needs to be preserved. */
    intr_register_int (14, 0, INTR_OFF, page_fault, "#PF Page-Fault Exception");
}
```

```
#ifndef THREADS_INTR_STUBS_H
#define THREADS_INTR_STUBS_H
```

```
/** Interrupt stubs.
```

```
These are little snippets of code in intr-stubs.S, one for
each of the 256 possible x86 interrupts.  Each one does a
```

```

    little bit of stack manipulation, then jumps to intr_entry().
    See intr-stubs.S for more information.

    This array points to each of the interrupt stub entry points
    so that intr_init() can easily find them. */
typedef void intr_stub_func (void);
extern intr_stub_func *intr_stubs[256];

/** Interrupt return path. */
void intr_exit (void);

#endif /**< threads/intr-stubs.h */

```

아래는 pintos에 구현된 intr-stubs.S의 코드이다.

```

...

.func intr_entry
intr_entry:
    /* Save caller's registers. */
    pushl %ds
    pushl %es
    pushl %fs
    pushl %gs
    pushal

    /* Set up kernel environment. */
    cld          /* String instructions go upward. */
    mov $SEL_KDSEG, %eax    /* Initialize segment registers. */
    mov %eax, %ds
    mov %eax, %es
    leal 56(%esp), %ebp /* Set up frame pointer. */

    /* Call interrupt handler. */
    pushl %esp
.globl intr_handler
    call intr_handler
    addl $4, %esp
.endfunc

```

위의 code implementation에서 intr_entry는 메인 interrupt entry point로 기능한다. 모든 인터럽트는 intrNN_stub라는 코드 조각에서 시작하며 interrupt number와 error code등의 정보를 stack에 push라고 intr_entry로 jump해서 오게된다. code는 어셈블리어로 구현되어 있으며 pushal 명령어로 caller의 레지스터 값을 스택에 푸시하여 저장한다. 이후 segment register를 커널 데이터 세그먼트로 설정하고 frame pointer를 조정하여 커널 환경 설정을 수행한다. 이후에는 실질적으로 인터럽트가 처리될 수 있도록 stack pointer를 push하고 intr_handler를 호출한다. 즉, system call이 발생하면 interrupt vector number 0x30이 발생하고 이 번호에 대한 intr_entry가 호출되어 memory access 이후 intr_handler가 호출되어 처리된다. 인터럽트 처리 이후의 수행은 아래의 코드에 구현되어 있다.


```

...
.globl intr_exit
.func intr_exit
intr_exit:
    /* Restore caller's registers. */
    popal
    popl %gs
    popl %fs
    popl %es
    popl %ds

    /* Discard `struct intr_frame' vec_no, error_code,
       frame_pointer members. */
    addl $12, %esp

    /* Return to caller. */
    iret
.endfunc

```

`intr_exit`은 syscall 인터럽트 처리 이후에 호출되며 저장된 caller의 레지스터값을 restore한다. (via popl) stack에서 추가로 저장된 인터럽트 관련 데이터를 제거한 후 `iret` instruction을 통해 원래의 call위치로 되돌아갈 수 있도록 한다. 즉, 다시말해 `iret`명령어는 일종의 interrupt return 명령으로 인터럽트 발생 이전의 user모드로 복귀할 수 있도록 한다.

```

.data
.globl intr_stubs
intr_stubs:

/* This implements steps 1 and 2, described above, in the common
   case where we just push a 0 error code. */
#define zero \
    pushl %ebp; \
    pushl $0

/* This implements steps 1 and 2, described above, in the case
   where the CPU already pushed an error code. */
#define REAL \
    pushl (%esp); \
    movl %ebp, 4(%esp)

/* Emits a stub for interrupt vector NUMBER.
   TYPE is `zero', for the case where we push a 0 error code,
   or `REAL', if the CPU pushes an error code for us. */
#define STUB(NUMBER, TYPE) \
    .text; \
    .func intr##NUMBER##_stub; \
    intr##NUMBER##_stub: \
        TYPE; \
        push $0x##NUMBER; \
        jmp intr_entry; \

```

```

.endfunc;

.data;
.long intr##NUMBER##_stub;

/* All the stubs. */
STUB(00, zero) STUB(01, zero) STUB(02, zero) STUB(03, zero)
...
/* 종락 */
...
STUB(fc, zero) STUB(fd, zero) STUB(fe, zero) STUB(ff, zero)

```

위의 구현은 stub 매크로 및 intrNN_stub에 대해 다루고 있으며 stub매크로는 0x00부터 0xff까지의 interrupt vector에 대한 stub을 정의한다. 각 stub은 해당 interrupt발생에 대해 앞서 설명한 intr_entry로의 jump이전의 initialization을 수행한다. intrNN_stub은 각 interrupt vector에 대한 entry point로서 stack에 frame_pointer, error code, vec_no와 같은 field를 push하여 처리할 수 있도록 하고 jmp명령어를 통해 intr_entry로 점프하여 인터럽트 처리가 가능하도록 한다. 이러한 코드 구현은 (예시로 syscall0의 경우) syscall0(NUMBER)에서 발생한 int \$0x30 interrupt에 대해 intr30_stub이 실행되어 intr_entry로의 점프가 이루어질 수 있도록 구현한다. intr_entry에서는 위에서의 설명과 같이 필요한 레지스터와 커널 환경 설정이 수행되고 실제 인터럽트가 처리되는 intr_handler가 호출되어 NUMBER와 적절한 argument를 전달하여 요구되는 필요한 kernel service가 수행될 수 있도록 한다.

이와 관련하여 userprog/syscall.c에 구현된 syscall_init을 살펴보겠다.

```

void
syscall_init (void)
{
    intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");
}

```

위의 코드 구현에서 intr_handlers[0x30]의 mapping은 syscall_handler임을 알 수 있다. 이로 인해 위에서 호출된 intr_handler의 경우 syscall_handler의 호출로 이어진다. 그러나 syscall_handler의 경우 앞선 구현 설명에서와 같이 메시지 출력과 함께 thread_exit으로만 구현되어 있어 적절한 number에 따른 kernel service가 제공 되도록 system call을 처리하는 기능 구현이 이루어지지 않았다.

File System

먼저, file system의 주요 구조체 세 가지에 대해 알아보았다. 첫 번째로, inode 구조체는 file system에서 파일이나 디렉토리 정보를 저장하는 역할을 한다. elem은 여러 inode를 연결한 리스트 요소이며, sector는 해당 inode가 디스크의 어떤 섹터에 저장되어 있는지를 나타내는 정수값이다. open_cnt는 현재 열려 있는 inode의 개수를 나타내며, 이는 파일이나 디렉토리가 몇 번 열려 있는지를 알려준다. removed는 해당 inode가 삭제되었는지 여부를 나타내며, deny_write_cnt는 파일에 대한 쓰기 작업 허용 여부를 관리하는 변수이다. 마지막으로, data는 아래에서 설명할 inode_disk 구조체를 가리키는 변수로, 실제 파일의 데이터를 저장한다.

```

struct inode
{

```

```

    struct list_elem elem;           /* Element in inode list. */
    block_sector_t sector;           /* Sector number of disk location. */
    int open_cnt;                    /* Number of openers. */
    bool removed;                    /* True if deleted, false otherwise. */
    int deny_write_cnt;              /* 0: writes ok, >0: deny writes. */
    struct inode_disk data;          /* Inode content. */
};

```

inode_disk 구조체는 앞서 언급한 대로 inode의 내용을 저장하는 구조체이다. start는 파일이 저장된 디스크의 첫 번째 섹터를 나타내며, 즉 파일이 디스크에서 시작하는 위치를 의미한다. length는 파일의 크기를 바이트 단위로 나타내며, 파일의 실제 크기를 표현한다. magic은 해당 구조체가 올바르게 저장되어 있는지를 확인하기 위한 "매직 넘버" 역할을 한다. 마지막으로, unused[125]는 구조체의 크기를 맞추기 위해 할당된 빈 배열이다.

```

struct inode_disk
{
    block_sector_t start;           /* First data sector. */
    off_t length;                  /* File size in bytes. */
    unsigned magic;                /* Magic number. */
    uint32_t unused[125];          /* Not used. */
};

```

세 번째 구조체는 file이다. 이 구조체는 파일의 inode를 가리키는 포인터, 현재 파일 포인터의 위치를 나타내는 pos, 그리고 file_deny_write 함수가 호출되었는지 여부를 확인하는 deny_write 변수를 포함하고 있다.

```

struct file
{
    struct inode *inode;           /* File's inode. */
    off_t pos;                    /* Current position. */
    bool deny_write;              /* Has file_deny_write() been called? */
};

```

먼저, 파일을 여는 함수인 file_open에 대해 설명하겠다. 파일을 새로 열 때, 앞서 설명한 새로운 file 객체를 생성하고 초기화해야 한다. calloc 함수를 사용하여 file 구조체의 크기만큼 메모리를 동적 할당하고, 모든 바이트를 0으로 초기화한다. 전달받은 inode를 생성한 file 구조체에 저장하고, pos(즉, 파일의 시작 위치)를 0으로 초기화한다. 그리고 deny_write는 아직 file_deny_write 함수가 호출되지 않았기 때문에 false로 설정한 후, 해당 file 객체를 반환한다.

그러나, 만약 전달받은 inode가 null이거나 file 객체를 생성하지 못한 경우, inode_close를 호출하여 해당 inode를 닫고, file 객체에 대한 메모리를 해제한다.

```

struct file *
file_open (struct inode *inode)
{
    struct file *file = calloc (1, sizeof *file);
    if (inode != NULL && file != NULL)
    {

```

```

        file->inode = inode;
        file->pos = 0;
        file->deny_write = false;
        return file;
    }
    else
    {
        inode_close (inode);
        free (file);
        return NULL;
    }
}

```

inode_close 함수에서는 먼저 전달받은 inode 객체가 비어있는지 확인한 후, inode가 참조된 횟수를 나타내는 open_cnt 값을 1 줄이고, 그 값이 0과 같은지 확인한다. 만약 open_cnt가 0이라면, 해당 inode를 elem 리스트에서 제거하고, inode의 removed 변수가 true일 경우, 즉 파일을 삭제해야 하는 경우, inode가 저장된 섹터의 메모리를 해제하고, 파일의 데이터가 저장되어 있는 메모리 블록도 해제해준다. 마지막으로, inode 자체도 메모리에서 해제한다.

```

void
inode_close (struct inode *inode)
{
    /* Ignore null pointer. */
    if (inode == NULL)
        return;

    /* Release resources if this was the last opener. */
    if (--inode->open_cnt == 0)
    {
        /* Remove from inode list and release lock. */
        list_remove (&inode->elem);

        /* Deallocate blocks if removed. */
        if (inode->removed)
        {
            free_map_release (inode->sector, 1);
            free_map_release (inode->data.start,
                              bytes_to_sectors (inode->data.length));
        }

        free (inode);
    }
}

```

파일을 다시 여는 함수에서는, 인자로 전달받은 파일의 inode를 inode_reopen 함수를 통해 먼저 연 다음, 이 inode를 기반으로 file_open 함수를 호출하여 새롭게 file 객체를 생성하고 반환한다. inode_reopen 함수를 살펴보면, 인자로 전달된 inode가 비어있지 않은 경우, 해당 inode의 open_cnt 값을 1 증가시킨 뒤, 그 inode를 반환한다.

```

struct file *
file_reopen (struct file *file)
{
    return file_open (inode_reopen (file->inode));
}

struct inode *
inode_reopen (struct inode *inode)
{
    if (inode != NULL)
        inode->open_cnt++;
    return inode;
}

```

파일을 닫는 함수에서는 먼저, 인자로 전달된 file 객체가 비어있는지 확인한 후, file_allow_write 함수를 호출하여 해당 파일에 대한 쓰기 제한을 해제한다. 이후, inode_close 함수를 통해 inode를 메모리에서 해제하고, 마지막으로 동적 할당했던 file 객체를 메모리에서 해제한다.

```

void
file_close (struct file *file)
{
    if (file != NULL)
    {
        file_allow_write (file);
        inode_close (file->inode);
        free (file);
    }
}

```

file_get_inode는 file 객체의 inode를 반환하는 함수이다.

```

struct inode *
file_get_inode (struct file *file)
{
    return file->inode;
}

```

파일에서 데이터를 읽을 때는 file_read 함수가 사용된다. 이 함수는 먼저 inode_read_at 함수를 통해 파일의 현재 위치인 file->pos에서부터 지정된 size만큼의 데이터를 buffer에 저장한다. 이후, pos에 읽어온 바이트 수만큼 더해주어 현재 위치를 새롭게 업데이트한 다음, 읽은 바이트 수를 반환한다.

```

off_t
file_read (struct file *file, void *buffer, off_t size)
{
    off_t bytes_read = inode_read_at (file->inode, buffer, size, file->pos);
    file->pos += bytes_read;
}

```

```

    return bytes_read;
}

```

file_read_at 함수는 특정 위치에서 데이터를 읽는 함수로, file_ofs에서 지정된 size만큼의 데이터를 읽어 buffer에 저장한 후, 읽은 바이트 수를 반환한다.

```

off_t
file_read_at (struct file *file, void *buffer, off_t size, off_t file_ofs)
{
    return inode_read_at (file->inode, buffer, size, file_ofs);
}

```

위에서 사용한 inode_read_at 함수는 섹터 단위로 데이터를 읽어와 필요한 만큼 버퍼에 복사하는 역할을 한다. size가 0보다 클 때, 즉 읽어야 할 데이터가 남아 있을 때, while 루프를 돌며 섹터 데이터를 버퍼에 저장한다. 이때, inode_left(inode 내에 남은 바이트 수)와 sector_left(섹터 내에 남은 바이트 수)를 비교하여 더 작은 값을 기준으로 실제로 읽어올 바이트 크기인 chunk_size를 결정한다. 섹터의 데이터를 가져올 때는 block_read 함수를 사용하며, 만약 섹터 전체를 복사해야 할 경우 block_read를 통해 버퍼에 직접 복사하고, 부분적으로 복사할 경우에는 먼저 버퍼가 메모리를 할당받았는지 확인한 후 block_read로 버퍼의 특정 위치에 복사해 준다. 모든 읽기 명령이 끝난 후에는 free 함수를 이용해 bounce buffer의 메모리 할당을 해제한다.

```

off_t
inode_read_at (struct inode *inode, void *buffer_, off_t size, off_t offset)
{
    uint8_t *buffer = buffer_;
    off_t bytes_read = 0;
    uint8_t *bounce = NULL;

    while (size > 0)
    {
        /* Disk sector to read, starting byte offset within sector. */
        block_sector_t sector_idx = byte_to_sector (inode, offset);
        int sector_ofs = offset % BLOCK_SECTOR_SIZE;

        /* Bytes left in inode, bytes left in sector, lesser of the two. */
        off_t inode_left = inode_length (inode) - offset;
        int sector_left = BLOCK_SECTOR_SIZE - sector_ofs;
        int min_left = inode_left < sector_left ? inode_left : sector_left;

        /* Number of bytes to actually copy out of this sector. */
        int chunk_size = size < min_left ? size : min_left;
        if (chunk_size <= 0)
            break;

        if (sector_ofs == 0 && chunk_size == BLOCK_SECTOR_SIZE)
        {
            /* Read full sector directly into caller's buffer. */
            block_read (fs_device, sector_idx, buffer + bytes_read);
        }
    }
}

```

```

    else
    {
        /* Read sector into bounce buffer, then partially copy
           into caller's buffer. */
        if (bounce == NULL)
        {
            bounce = malloc (BLOCK_SECTOR_SIZE);
            if (bounce == NULL)
                break;
        }
        block_read (fs_device, sector_idx, bounce);
        memcpy (buffer + bytes_read, bounce + sector_ofs, chunk_size);
    }

    /* Advance. */
    size -= chunk_size;
    offset += chunk_size;
    bytes_read += chunk_size;
}
free (bounce);

return bytes_read;
}

```

아래는 네 개의 쓰기 관련 함수들이다. 먼저, `file_write` 함수는 `inode_write_at` 함수를 이용하여 `file->pos`(현재 위치)에서 `size`만큼 `buffer`에 있는 데이터를 쓴다. 이후, 쓴 바이트 수만큼 `pos`에 더해 현재 위치를 업데이트하고, 최종적으로 쓴 바이트 수를 반환한다.

```

off_t
file_write (struct file *file, const void *buffer, off_t size)
{
    off_t bytes_written = inode_write_at (file->inode, buffer, size, file->pos);
    file->pos += bytes_written;
    return bytes_written;
}

```

`file_write_at` 함수는 위에서 설명한 `file_read_at` 함수처럼 특정 위치 (`file_ofs`)에서 쓰기 작업을 수행하는 함수이다.

```

off_t
file_write_at (struct file *file, const void *buffer, off_t size,
               off_t file_ofs)
{
    return inode_write_at (file->inode, buffer, size, file_ofs);
}

```

`inode_write_at` 함수 역시 앞서 설명한 `inode_read_at` 함수와 유사하게, `size`에 따라 `while` 루프를 돌며 `block_write` 함수를 통해 쓰기 작업을 처리한다.

```

off_t
inode_write_at (struct inode *inode, const void *buffer_, off_t size,
                off_t offset)
{
    const uint8_t *buffer = buffer_;
    off_t bytes_written = 0;
    uint8_t *bounce = NULL;

    if (inode->deny_write_cnt)
        return 0;

    while (size > 0)
    {
        /* Sector to write, starting byte offset within sector. */
        block_sector_t sector_idx = byte_to_sector (inode, offset);
        int sector_ofs = offset % BLOCK_SECTOR_SIZE;

        /* Bytes left in inode, bytes left in sector, lesser of the two. */
        off_t inode_left = inode_length (inode) - offset;
        int sector_left = BLOCK_SECTOR_SIZE - sector_ofs;
        int min_left = inode_left < sector_left ? inode_left : sector_left;

        /* Number of bytes to actually write into this sector. */
        int chunk_size = size < min_left ? size : min_left;
        if (chunk_size <= 0)
            break;

        if (sector_ofs == 0 && chunk_size == BLOCK_SECTOR_SIZE)
        {
            /* Write full sector directly to disk. */
            block_write (fs_device, sector_idx, buffer + bytes_written);
        }
        else
        {
            /* We need a bounce buffer. */
            if (bounce == NULL)
            {
                bounce = malloc (BLOCK_SECTOR_SIZE);
                if (bounce == NULL)
                    break;
            }

            /* If the sector contains data before or after the chunk
               we're writing, then we need to read in the sector
               first. Otherwise we start with a sector of all zeros. */
            if (sector_ofs > 0 || chunk_size < sector_left)
                block_read (fs_device, sector_idx, bounce);
            else
                memset (bounce, 0, BLOCK_SECTOR_SIZE);
            memcpy (bounce + sector_ofs, buffer + bytes_written, chunk_size);
            block_write (fs_device, sector_idx, bounce);
        }
    }
}

```



```

        /* Advance. */
        size -= chunk_size;
        offset += chunk_size;
        bytes_written += chunk_size;
    }
    free (bounce);

    return bytes_written;
}

```

아래의 두 함수는 파일에 대한 쓰기 작업을 금지하거나 허가하는 함수이다. 먼저, `file_deny_write` 함수는 `file` 객체의 `deny_write` 변수가 `false`일 경우 이를 `true`로 설정한 다음, `inode_deny_write` 함수를 호출하여 해당 `inode`에 대한 쓰기를 금지한다. `inode_deny_write` 함수에서는 먼저 해당 `inode`의 `deny_write_cnt`를 1 증가시킨 뒤, 이 값이 `open_cnt` 이하인지 확인한다. 이는 쓰기를 제한하려면 해당 파일이 먼저 열려 있어야 하므로, 쓰기를 제한하는 파일의 개수보다 열려 있는 개수가 더 클 수 없다는 조건을 확인하는 것이다.

```

void
file_deny_write (struct file *file)
{
    ASSERT (file != NULL);
    if (!file->deny_write)
    {
        file->deny_write = true;
        inode_deny_write (file->inode);
    }
}

void
inode_deny_write (struct inode *inode)
{
    inode->deny_write_cnt++;
    ASSERT (inode->deny_write_cnt <= inode->open_cnt);
}

```

반대로, `file_allow_write` 함수에서는 쓰기 작업을 허가해야 하므로, `deny_write`가 `true`인 경우 이를 `false`로 설정한 후, `inode_allow_write` 함수를 호출하여 해당 `inode`에 대한 쓰기를 허가한다. `inode_allow_write` 함수에서는 먼저 해당 `inode`의 `deny_write_cnt`가 0보다 큰지 확인한다. `deny_write_cnt` 변수가 0보다 크다는 것은 쓰기가 금지되어 있음을 의미한다. 또한, 앞서와 동일한 이유로, 해당 `inode`의 `deny_write_cnt`가 `open_cnt`보다 작거나 같은지 확인한다. 마지막으로, `deny_write_cnt` 값을 1 감소시켜 쓰기 제한을 해제한다.

```

void
file_allow_write (struct file *file)
{
    ASSERT (file != NULL);
    if (file->deny_write)
    {
        file->deny_write = false;
    }
}

```

```

        inode_allow_write (file->inode);
    }
}

void
inode_allow_write (struct inode *inode)
{
    ASSERT (inode->deny_write_cnt > 0);
    ASSERT (inode->deny_write_cnt <= inode->open_cnt);
    inode->deny_write_cnt--;
}

```

file_length 함수는 먼저, 인자로 전달된 file 객체가 비어있는지 확인한 후, inode_length 함수를 호출하여 파일의 길이를 반환한다. inode_length 함수는 인자로 전달된 inode의 data에 있는 length 값을 반환한다.

```

off_t
file_length (struct file *file)
{
    ASSERT (file != NULL);
    return inode_length (file->inode);
}

off_t
inode_length (const struct inode *inode)
{
    return inode->data.length;
}

```

file_seek 함수는 파일의 위치 포인터인 pos를 인자로 받은 new_pos로 재설정하는 함수이다. 먼저, 인자로 전달된 file 객체가 비어있는지 확인하고, 새로운 위치 new_pos가 음수가 아닌 유효한 값인지 검사한다. 이후, 유효한 값일 경우 file->pos를 new_pos로 설정해준다.

```

void
file_seek (struct file *file, off_t new_pos)
{
    ASSERT (file != NULL);
    ASSERT (new_pos >= 0);
    file->pos = new_pos;
}

```

file_tell 함수는 인자로 전달된 file 객체가 비어있지 않을 경우, 해당 file의 포인터 위치인 pos를 반환해준다.

```

off_t
file_tell (struct file *file)
{
    ASSERT (file != NULL);
}

```

```
    return file->pos;
}
```

fileSYS_init 함수는 파일 시스템을 초기화하는 함수로, 먼저 inode_init과 free_map_init 함수를 호출하여 파일 시스템을 초기화한다. 이후, 아래에서 설명할 do_format 함수를 통해 파일 시스템을 포맷하고 새로 설정한다. 마지막으로, free_map_open 함수를 호출하여 free_map을 연다.

```
void
fileSYS_init (bool format)
{
    fs_device = block_get_role (BLOCK_FILESYS);
    if (fs_device == NULL)
        PANIC ("No file system device found, can't initialize file system.");

    inode_init ();
    free_map_init ();

    if (format)
        do_format ();

    free_map_open ();
}
```

inode_init 함수를 보면 list_init 함수를 사용하여 open_inodes 리스트의 헤더를 설정해줌으로써 open_inodes list를 초기화해준다.

```
void
inode_init (void)
{
    list_init (&open_inodes);
}
```

여기서 말하는 free map은 파일 시스템에서 사용되지 않은 섹터를 찾기 위한 비트맵(bitmap)이다. 각 비트는 하나의 섹터를 나타내며, 해당 섹터의 사용 여부를 알려준다. free_map_init 함수는 free map을 초기화하는 역할을 한다. 그리고 free_map_open 함수는 디스크에 저장된 free_map 파일을 열어 free_map_file에 저장한다.

```
void
free_map_init (void)
{
    free_map = bitmap_create (block_size (fs_device));
    if (free_map == NULL)
        PANIC ("bitmap creation failed--file system device is too large");
    bitmap_mark (free_map, FREE_MAP_SECTOR);
    bitmap_mark (free_map, ROOT_DIR_SECTOR);
}
```

```

void
free_map_open (void)
{
    free_map_file = file_open (inode_open (FREE_MAP_SECTOR));
    if (free_map_file == NULL)
        PANIC ("can't open free map");
    if (!bitmap_read (free_map, free_map_file))
        PANIC ("can't read free map");
}

```

inode_open 함수는 인자로 받은 섹터에 있는 inode를 여는 역할을 한다. 만약 이미 열려 있는 inode가 있다면, inode_reopen 함수를 통해 다시 열어주고, 그렇지 않으면 새로운 inode를 malloc을 통해 메모리를 할당하여 초기화한다. 이때, open_cnt는 1로 설정하여 현재 열려 있는 inode가 해당 inode 하나뿐임을 나타내고, deny_write_cnt는 0으로 초기화하여 쓰기 작업이 가능하도록 한다. 이후, block_read 함수를 사용해 해당 섹터의 내용을 읽어와 inode->data에 저장한다. 마지막으로, 해당 inode를 반환한다.

```

struct inode *
inode_open (block_sector_t sector)
{
    struct list_elem *e;
    struct inode *inode;

    /* Check whether this inode is already open. */
    for (e = list_begin (&open_inodes); e != list_end (&open_inodes);
         e = list_next (e))
    {
        inode = list_entry (e, struct inode, elem);
        if (inode->sector == sector)
        {
            inode_reopen (inode);
            return inode;
        }
    }

    /* Allocate memory. */
    inode = malloc (sizeof *inode);
    if (inode == NULL)
        return NULL;

    /* Initialize. */
    list_push_front (&open_inodes, &inode->elem);
    inode->sector = sector;
    inode->open_cnt = 1;
    inode->deny_write_cnt = 0;
    inode->removed = false;
    block_read (fs_device, inode->sector, &inode->data);
    return inode;
}

```

do_format 함수는 파일 시스템을 포맷하는 함수이다. 먼저, free_map_create 함수를 호출하여 새로운 free map을 생성하고, dir_create 함수를 이용해 파일 시스템의 시작 지점에 16개의 엔트리를 갖는 디렉토리를 생성한다. 이후, free_map_close 함수를 호출하여 free map을 닫는다.

```
static void
do_format (void)
{
    printf ("Formatting file system...");
    free_map_create ();
    if (!dir_create (ROOT_DIR_SECTOR, 16))
        PANIC ("root directory creation failed");
    free_map_close ();
    printf ("done.\n");
}
```

free_map_create 함수는 inode_create 함수를 통해 새로운 inode를 생성한 후, file_open 함수를 사용해 해당 파일을 연다. 이후, bitmap_write를 통해 비트맵을 파일에 기록한다.

```
void
free_map_create (void)
{
    /* Create inode. */
    if (!inode_create (FREE_MAP_SECTOR, bitmap_file_size (free_map)))
        PANIC ("free map creation failed");

    /* Write bitmap to file. */
    free_map_file = file_open (inode_open (FREE_MAP_SECTOR));
    if (free_map_file == NULL)
        PANIC ("can't open free map");
    if (!bitmap_write (free_map, free_map_file))
        PANIC ("can't write free map");
}
```

inode_create 함수는 지정된 섹터에 length를 길이로 갖는 파일을 포함한 inode를 생성하고 초기화한다. 먼저, inode_disk 구조체를 가리키는 포인터를 생성하고 초기화한다. 이후, inode_disk 구조체에 calloc을 사용해 동적으로 메모리를 할당하고, 모든 바이트를 0으로 초기화한다. length에 맞게 필요한 섹터의 수를 bytes_to_sectors 함수를 통해 계산한 후, inode의 길이를 설정한다. 다음으로, free_map_allocate 함수를 통해 섹터를 할당하고 inode 데이터를 저장한다. 마지막으로, inode_disk의 메모리를 해제하고, 성공 여부를 반환한다.

```
bool
inode_create (block_sector_t sector, off_t length)
{
    struct inode_disk *disk_inode = NULL;
    bool success = false;

    ASSERT (length >= 0);
```

```

/* If this assertion fails, the inode structure is not exactly
   one sector in size, and you should fix that. */
ASSERT (sizeof *disk_inode == BLOCK_SECTOR_SIZE);

disk_inode = calloc (1, sizeof *disk_inode);
if (disk_inode != NULL)
{
    size_t sectors = bytes_to_sectors (length);
    disk_inode->length = length;
    disk_inode->magic = INODE_MAGIC;
    if (free_map_allocate (sectors, &disk_inode->start))
    {
        block_write (fs_device, sector, disk_inode);
        if (sectors > 0)
        {
            static char zeros[BLOCK_SECTOR_SIZE];
            size_t i;

            for (i = 0; i < sectors; i++)
                block_write (fs_device, disk_inode->start + i, zeros);
        }
        success = true;
    }
    free (disk_inode);
}
return success;
}

```

free_map_close 함수는 열려 있는 free_map 파일을 닫는 역할을 한다.

```

void
free_map_close (void)
{
    file_close (free_map_file);
}

```

dir_create 함수는 새로운 디렉토리를 생성하는 함수로, inode_create 함수를 호출하여 entry_cnt * sizeof(struct dir_entry)를 length로 하는 inode를 생성한다.

```

bool
dir_create (block_sector_t sector, size_t entry_cnt)
{
    return inode_create (sector, entry_cnt * sizeof (struct dir_entry));
}

```

filesystem_done 함수는 파일 시스템을 종료하는 함수로, free_map_close 함수를 호출하여 사용한 리소스들을 정리 해준다.

```
void
filesys_done (void)
{
    free_map_close ();
}
```

`filesys_create` 함수는 파일을 생성하고 디렉토리에 추가하는 역할을 한다. 먼저, `dir_open_root` 함수를 호출하여 루트 디렉토리의 위치를 `dir`에 포인터로 저장한다. 이후, `free_map_allocate` 함수를 통해 새 파일을 위한 블록을 할당하고, 해당 섹터 번호를 `inode_sector`에 저장한다. `inode_create` 함수를 사용하여 새로운 `inode`를 생성한 후, `dir_add` 함수를 통해 해당 파일을 디렉토리에 추가한다. 위의 세 가지 작업 중 하나라도 실패하거나 `inode_sector`가 0이 아닐 경우, 즉 이미 블록이 할당된 상태라면, 할당된 블록을 `free_map_release` 함수를 통해 해제한다. 마지막으로, 루트 디렉토리를 닫고 파일 생성의 성공 여부를 반환한다.

```
bool
filesys_create (const char *name, off_t initial_size)
{
    block_sector_t inode_sector = 0;
    struct dir *dir = dir_open_root ();
    bool success = (dir != NULL
                    && free_map_allocate (1, &inode_sector)
                    && inode_create (inode_sector, initial_size)
                    && dir_add (dir, name, inode_sector));
    if (!success && inode_sector != 0)
        free_map_release (inode_sector, 1);
    dir_close (dir);

    return success;
}
```

`dir_open_root`는 `dir_open` 함수를 사용하여 루트 디렉토리에 해당하는 `inode`(디렉토리 구조체) 전체를 반환하므로, `dir_open` 함수를 이해해야 한다. `dir_open` 함수는 `pos`를 0으로 설정하고, 주어진 `inode`를 기반으로 한 디렉토리 구조체를 반환한다.

```
struct dir *
dir_open (struct inode *inode)
{
    struct dir *dir = calloc (1, sizeof *dir);
    if (inode != NULL && dir != NULL)
    {
        dir->inode = inode;
        dir->pos = 0;
        return dir;
    }
    else
    {
        inode_close (inode);
    }
}
```

```

        free (dir);
        return NULL;
    }
}

struct dir *
dir_open_root (void)
{
    return dir_open (inode_open (ROOT_DIR_SECTOR));
}

```

free_map_allocate 함수는 cnt만큼 연속적으로 비어 있는 섹터를 찾아 해당 섹터들의 비트를 true로 설정하여 할당하는 함수이다. 이후, 찾은 섹터의 값을 sectorp(섹터 포인터)에 저장하고, 할당의 성공 여부를 나타내는 불리언 값을 반환한다.

```

bool
free_map_allocate (size_t cnt, block_sector_t *sectorp)
{
    block_sector_t sector = bitmap_scan_and_flip (free_map, 0, cnt, false);
    if (sector != BITMAP_ERROR
        && free_map_file != NULL
        && !bitmap_write (free_map, free_map_file))
    {
        bitmap_set_multiple (free_map, sector, cnt, false);
        sector = BITMAP_ERROR;
    }
    if (sector != BITMAP_ERROR)
        *sectorp = sector;
    return sector != BITMAP_ERROR;
}

```

free_map_release 함수는 bitmap_all 함수를 사용해 범위 내의 모든 섹터가 할당된 상태인지 확인한 후, bit_set_multiple 함수를 통해 해당 섹터들의 비트를 0으로 설정한다. 이후, 변경사항을 bitmap_write 함수를 통해 free_map_file에 업데이트한다.

```

void
free_map_release (block_sector_t sector, size_t cnt)
{
    ASSERT (bitmap_all (free_map, sector, cnt));
    bitmap_set_multiple (free_map, sector, cnt, false);
    bitmap_write (free_map, free_map_file);
}

```

dir_add 함수는 디렉토리에 새로운 엔트리를 추가하는 역할을 한다. for 루프를 통해 빈 디렉토리 엔트리를 찾은 후, 빈 엔트리가 있다면 새로운 엔트리를 추가한다. 이후, 추가 과정의 성공 여부를 반환한다.


```

bool
dir_add (struct dir *dir, const char *name, block_sector_t inode_sector)
{
    struct dir_entry e;
    off_t ofs;
    bool success = false;

    ASSERT (dir != NULL);
    ASSERT (name != NULL);

    /* Check NAME for validity. */
    if (*name == '\\0' || strlen (name) > NAME_MAX)
        return false;

    /* Check that NAME is not in use. */
    if (lookup (dir, name, NULL, NULL))
        goto done;

    /* Set OFS to offset of free slot.
       If there are no free slots, then it will be set to the
       current end-of-file.

       inode_read_at() will only return a short read at end of file.
       Otherwise, we'd need to verify that we didn't get a short
       read due to something intermittent such as low memory. */
    for (ofs = 0; inode_read_at (dir->inode, &e, sizeof e, ofs) == sizeof e;
        ofs += sizeof e)
        if (!e.in_use)
            break;

    /* Write slot. */
    e.in_use = true;
    strcpy (e.name, name);
    e.inode_sector = inode_sector;
    success = inode_write_at (dir->inode, &e, sizeof e, ofs) == sizeof e;

done:
    return success;
}

```

`filesys_open` 함수에서는 `dir_lookup` 함수를 사용하여 `name`이라는 파일 이름을 가진 파일을 디렉토리에
서 찾은 후, 해당 파일의 `inode`를 기반으로 `file_open` 함수를 호출하여 `file` 객체를 생성하고 반환한다.

```

struct file *
filesys_open (const char *name)
{
    struct dir *dir = dir_open_root ();
    struct inode *inode = NULL;

    if (dir != NULL)

```

```

    dir_lookup (dir, name, &inode);
    dir_close (dir);

    return file_open (inode);
}

```

dir_lookup 함수는 inode_open 함수를 사용하여 해당 섹터에 있는 inode를 연 후, 성공 여부를 반환한다. 여기서 말하는 inode는 디렉토리의 엔트리로 name을 갖는 inode를 의미한다.

```

bool
dir_lookup (const struct dir *dir, const char *name,
            struct inode **inode)
{
    struct dir_entry e;

    ASSERT (dir != NULL);
    ASSERT (name != NULL);

    if (lookup (dir, name, &e, NULL))
        *inode = inode_open (e.inode_sector);
    else
        *inode = NULL;

    return *inode != NULL;
}

```

dir_close 함수는 inode_close와 free 함수를 사용하여 열려 있는 디렉토리를 닫고, 메모리를 해제하는 역할을 한다.

```

void
dir_close (struct dir *dir)
{
    if (dir != NULL)
    {
        inode_close (dir->inode);
        free (dir);
    }
}

```

fileys_remove 함수는 name이라는 파일 이름을 가진 파일을 디렉토리에서 제거하는 역할을 한다. 이 함수는 dir_remove 함수를 사용하여 파일을 삭제하고, 그 성공 여부를 반환한다.

```

bool
fileys_remove (const char *name)
{
    struct dir *dir = dir_open_root ();
}

```

```

    bool success = dir != NULL && dir_remove (dir, name);
    dir_close (dir);

    return success;
}

```

dir_remove 함수는 name을 갖는 디렉토리 엔트리를 삭제하는 역할을 한다. dir_lookup 함수와 유사하게, lookup 함수를 통해 해당 엔트리를 찾고 inode를 연 후, inode_write_at 함수를 사용해 디렉토리 엔트리 삭제 상태를 업데이트한다. 이후, inode_remove와 inode_close 함수를 통해 해당 inode를 삭제하고 닫아준다. 마지막으로, 삭제 작업의 성공 여부를 반환한다.

```

bool
dir_remove (struct dir *dir, const char *name)
{
    struct dir_entry e;
    struct inode *inode = NULL;
    bool success = false;
    off_t ofs;

    ASSERT (dir != NULL);
    ASSERT (name != NULL);

    /* Find directory entry. */
    if (!lookup (dir, name, &e, &ofs))
        goto done;

    /* Open inode. */
    inode = inode_open (e.inode_sector);
    if (inode == NULL)
        goto done;

    /* Erase directory entry. */
    e.in_use = false;
    if (inode_write_at (dir->inode, &e, sizeof e, ofs) != sizeof e)
        goto done;

    /* Remove inode. */
    inode_remove (inode);
    success = true;

done:
    inode_close (inode);
    return success;
}

```

Design Implementation

1. Process Termination Messages

유저 프로세스가 종료될 때마다 `printf ("%s: exit(%d)\n", process_name, exit_code);` 코드를 추가하여 프로세스 종료 메시지를 출력해야 한다. 이를 위해, `thread` 구조체에 종료 메시지 출력 여부를 알려주는 `boolean` 변수를 추가할 예정이다. 새로운 프로세스가 `process_execute` 함수에서 생성될 경우 이 변수를 `true`로 설정하고, `halt`나 커널 스레드 종료 시에는 `false`로 설정한다. 이후 `thread_exit` 함수에서 해당 출력 명령을 수행하도록 한다.

2. Argument Passing

`process_execute` 함수는 현재 `"file_name"`으로 문자열 전체를 받아오지만, 이 문자열을 파일 이름과 인자로 구분하는 과정이 필요하다. `/src/lib/string.c`의 `strtok_r()` 함수를 사용하여 공백으로 구분하여 파일 이름과 인자를 분리하고, `start_process`를 호출할 때 스택에 인자들을 추가한 후 스택을 전달하는 방식으로 구현할 예정이다. 이를 위해 파싱된 인자들을 스택에 추가하는 함수를 만들고, 이 함수를 `start_process` 함수에서 성공 시(`if (success) {...}`) 실행되도록 수정할 예정이다.

```
char *
strtok_r (char *s, const char *delimiters, char **save_ptr)
{
    char *token;

    ASSERT (delimiters != NULL);
    ASSERT (save_ptr != NULL);

    /* If S is nonnull, start from it.
       If S is null, start from saved position. */
    if (s == NULL)
        s = *save_ptr;
    ASSERT (s != NULL);

    /* Skip any DELIMITERS at our current position. */
    while (strchr (delimiters, *s) != NULL)
    {
        /* strchr() will always return nonnull if we're searching
           for a null byte, because every string contains a null
           byte (at the end). */
        if (*s == '\0')
        {
            *save_ptr = s;
            return NULL;
        }

        s++;
    }

    /* Skip any non-DELIMITERS up to the end of the string. */
    token = s;
    while (strchr (delimiters, *s) == NULL)
        s++;
    if (*s != '\0')
    {
        *s = '\0';
        *save_ptr = s + 1;
    }
}
```

```

else
    *save_ptr = s;
return token;
}

```

3. System Call

현재 `syscall_handler`는 바로 종료하는 방식으로 구현되어 있으므로, 이 부분을 우선적으로 수정해야 한다. 우선 `intr_frame`의 `esp` 값을 읽어와, 스택에 사용자 프로그램이 `push`한 인자들과 시스템 호출 번호를 4바이트 단위로 읽어들이 이를 스택 또는 큐에 복사해 저장한다. 이후 시스템 호출에 필요한 정보를 `syscall_handler`에 저장하여 처리할 수 있도록 한다.

그 후 `syscall_handler`에 각 시스템 호출 번호에 해당하는 헬퍼 함수를 정의하고 구현하여, 각 번호에 맞는 시스템 호출이 적절한 헬퍼 함수에서 처리되도록 구현한다. 예를 들어, `sys_exit_helper`와 같은 헬퍼 함수를 통해 시스템 호출 번호와 인자를 전달하고, 반환된 값을 `intr_frame`의 `eax`에 저장할 수 있도록 한다. 이때 적절한 헬퍼함수의 구현은 아래 4번 file system관련해서 구현해야하는 함수와 중복된다. 예를들어 file creation과 관련된 `syscall`을 수행하는 함수의 경우 파일 시스템 함수를 그대로 사용하면 될것이다.

또한, 시스템 호출 및 실행 파일 쓰기 제한을 위한 `pcb`(프로세스 제어 블록) 구조체를 새롭게 정의하고자 한다. `pcb`에는 프로세스의 고유 ID, 부모 프로세스, 파일 시스템 접근을 위한 플래그 변수, 대기 처리를 위한 변수, 현재 실행 중인 파일이 포함되어야 한다. 그리고 `thread` 구조체에 이 `pcb` 포인터를 추가한 뒤, 프로세스가 생성되고 시작될 때(`start_process` 시점)에 해당 스레드의 `pcb`를 초기화할 예정이다. 여기서 추가된 현재 실행 중인 파일을 가리키는 포인터는 아래의 4번 과제에서 더 자세히 설명할 예정이다.

4. Denying Writes to Executables

현재 실행 중인 파일 각각에 대한 쓰기 허용/제한을 관리하는 방법으로 두 가지를 고려해 보았다. 첫 번째 방법은, 현재 실행 중인 파일들의 리스트를 만들어 각 파일의 실행이 끝날 때마다 리스트에서 제거하는 방식이다. 이 방법은 실행 중인 파일 목록을 직관적으로 파악할 수 있다는 장점이 있지만, 실행 종료 후 리스트에서 제거하거나 쓰기 작업을 위해 확인할 때 리스트의 모든 요소를 검사해야 하므로 오버헤드가 클 것으로 예상된다.

두 번째 방법은, 현재 디자인 분석에서 설명한 `file_deny_write`와 `file_allow_write` 함수를 활용하는 방식이다. 각 프로세스가 시작될 때 `file_deny_write` 함수를 호출하여 쓰기를 제한하고, 해당 프로세스가 종료되면 `file_allow_write` 함수를 호출하여 쓰기를 허용한다. 이번 프로젝트에서는 두 번째 방법을 이용하여 과제를 구현하고자 한다.

따라서, 현재 실행 중인 파일을 가리키는 포인터 변수를 위에서 생성한 `process control block` 구조체에 추가해 준다. 이후 `start_process`에서 `load` 함수를 호출한 후, `load` 함수 내에서 이 포인터가 현재 실행 중인 파일을 가리키도록 설정하고 `file_deny_write` 함수를 호출하여 쓰기를 제한한다. 마지막으로, 작업이 완료되어 `process_exit` 함수가 호출될 때 `file_allow_write` 함수를 호출하여 다시 쓰기 작업을 허용함으로써 과제 4번을 구현할 수 있을 것이다.